PES-VCS Lab Report

Building a Version Control System from Scratch

Part 1 — Implementation Screenshots

1A — ./test_objects: All Phase 1 Tests Passing

```
sudhanva@sudhanvas-MacBook-Air os-orange % ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Blob storage, deduplication, and integrity check all pass.

1B — find .pes/objects -type f: Sharded Directory Structure

```
sudhanva@sudhanvas-MacBook-Air os-orange % find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Objects sharded by first 2 hex chars of SHA-256 hash.

2A — ./test_tree: All Phase 2 Tests Passing

```
sudhanva@sudhanvas-MacBook-Air os-orange % ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Tree serialize/parse roundtrip and deterministic serialization verified.

2B — xxd of Raw Tree Object (first 20 lines)

```
sudhanva@sudhanvas-MacBook-Air os-orange % xxd .pes/objects/2a/594d39232787fba8e
b7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Binary format: 'blob \0' header visible in hex dump.

3A — pes init → pes add → pes status Sequence (manual)

```
ex
sudhanva@sudhanvas-MacBook-Air os-orange % ./pes status
error: failed to load index
sudhanva@sudhanvas-MacBook-Air os-orange % ./pes init
./pes add file1.txt  ./pes status
Initialized empty PES repository in .pes/
zsh: segmentation fault  ./pes add file1.txt ./pes status
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Segfault on pes add indicates index_add still under development at this stage.

3A (continued) — ./test_sequence.sh Integration Flow

```
sudhanva@sudhanvas-MacBook-Air os-orange % ./test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
./test_sequence.sh: line 36: 50537 Segmentation fault: 11  $PES add file.txt he
lo.txt
sudhanva@sudhanvas-MacBook-Air os-orange %
```

Init passes; segfault on add during automated test sequence.

3B — cat .pes/index: Text-Format Staging Area

```
sudhanva@sudhanvas-MacBook-Air os-orange % cat .pes/index
file1.txt dummyhash
```

Index file stores filename and blob hash in plain text.

4B — find .pes -type f | sort: Object Store Growth

```
sudhanva@sudhanvas-MacBook-Air os-orange % find .pes -type f | sort
.pes/HEAD
.pes/index
sudhanva@sudhanvas-MacBook-Air os-orange %
```

HEAD and index present after init; objects directory fills as commits are made.

4C — cat .pes/refs/heads/main and cat .pes/HEAD

```
sudhanva@sudhanvas-MacBook-Air os-orange % cat .pes/refs/heads/main
cat .pes/HEAD
cat: .pes/refs/heads/main: No such file or directory
ref: refs/heads/main
sudhanva@sudhanvas-MacBook-Air os-orange %
```

HEAD contains 'ref: refs/heads/main'; refs/heads/main not yet written (no commits).

Part 2 — Analysis Questions

Phases 5 & 6 cover filesystem concepts beyond implementation. Answers are written below.

Section 5 — Branching and Checkout

Q5.1 — Implementing pes checkout <branch>

A branch is just a file in *.pes/refs/heads/* whose content is a commit hash. Checkout must perform three steps:

1. **Resolve the target branch.** Read *.pes/refs/heads/<branch>* to get the target commit hash. If the file does not exist, abort with an error.

2. **Update .pes/HEAD.** Overwrite HEAD with the symbolic ref: *ref: refs/heads/<branch>*. This is the only metadata change.

3. **Update the working directory.** Walk the target commit's tree recursively and, for each entry, compare it with the current HEAD's tree. Files that differ are written from the object store; files that exist in current HEAD but not in the target tree are deleted; new files are created. The index is then rebuilt to match the target tree exactly.

What makes this complex: (a) *Dirty-working-directory detection* — if a tracked file has local modifications that would be overwritten, checkout must refuse to proceed (see Q5.2). (b) *Atomicity* — a crash mid-checkout leaves the repo in a mixed state; real Git uses a lock file and staged application. (c) *Untracked files* — files in the target tree that already exist untracked in the working directory would be silently overwritten, so checkout must check for collisions. (d) *Sub-tree depth* — nested directories must be created or pruned recursively, which means tracking which directories become empty and should be removed.

Q5.2 — Detecting a Dirty Working Directory Conflict

The algorithm uses only the index and the object store — no diff library required:

Step 1 — **Find files that differ between the two branches.** Parse the current HEAD commit's tree and the target branch's tree. For every path that appears in both trees with a different blob hash, that file *will change* during checkout.

Step 2 — **Check whether those files are clean in the working directory.** For each such path, look up its entry in the index. The index stores the blob hash that was last staged. Re-hash the actual file on disk (or compare mtime/size as a fast shortcut). If the on-disk content does not match the index entry, the file has been modified since the last *pes add* — it is dirty. Alternatively, if the on-disk hash differs from the current HEAD's blob hash for that path (even if the index is up to date), the file is also considered dirty.

Step 3 — **Refuse if any conflicting file is dirty.** Report each conflicting path and abort. The user must commit or discard changes first. Only if every path that would be overwritten is clean (on-disk == current HEAD blob) is it safe to proceed.

Q5.3 — Detached HEAD State

What detached HEAD means: Normally HEAD contains a symbolic ref (*ref: refs/heads/main*). In detached HEAD state, HEAD contains a raw commit hash directly, e.g. *a1b2c3d4...*. This happens when you checkout a specific commit rather than a branch name.

What happens if you commit in this state: New commits are created and chained correctly — each commit has the previous one as its parent. However, because HEAD holds a hash and not a branch name, *head_update*

writes the new commit hash directly into HEAD rather than into a branch ref file. Those commits are therefore not reachable from any branch. They exist in the object store but will be collected by garbage collection once HEAD moves elsewhere.

How to recover: The user can recover the commits as long as they know the hash of the last commit made in detached HEAD (the terminal shows it after each commit). The recovery procedure is: (1) Create a new branch pointing to that hash: *pes branch recovery-branch <hash>* (which writes the hash into *.pes/refs/heads/recovery-branch*). (2) Checkout that branch. Once a branch ref points to those commits, they are reachable and safe from GC. In real Git, the *reflog* records every HEAD movement, giving a safety net for recovery even without knowing the hash.

Section 6 — Garbage Collection and Space Reclamation

Q6.1 — Algorithm to Find and Delete Unreachable Objects

Algorithm — mark-and-sweep over the object graph:

Mark phase (reachability traversal): Start from every reference tip — every file under *.pes/refs/* and the current HEAD. For each starting commit hash, do a depth-first traversal: (a) add the commit hash to a *reachable* set; (b) parse the commit to get its tree hash and parent hash(es); (c) recursively mark the tree and all blobs/subtrees it references; (d) follow parent pointers until a commit with no parent is reached. Continue for every branch tip.

Sweep phase (deletion): Enumerate every file under *.pes/objects/* (reconstructing each hash from the two-char directory prefix + filename). For any hash not in the *reachable* set, delete the file. Optionally remove empty shard directories afterwards.

Data structure: A hash set (e.g. a C hash table or a sorted array of 32-byte hashes) for O(1) membership tests during the sweep. Memory usage scales with the number of reachable objects, not total objects.

Estimate for 100,000 commits, 50 branches: Assume each commit references a root tree with ~20 entries and 3 levels of subtrees; that is roughly 1 commit + 5 tree objects + 20 blob objects = ~26 objects per commit. With 100,000 commits, reachable objects total approximately 2.6 million. All 50 branch tips converge on shared history, so most objects are visited once. Total objects to visit during the mark phase: on the order of 2–3 million. Many blobs will be shared across commits (deduplication), so the actual object store may contain fewer than 2.6 million files.

Q6.2 — Race Condition Between GC and a Concurrent Commit

The race condition: Consider the following interleaving:

1. A writer process begins *pes commit*. It calls *object_write* for a new blob (file content). The blob is now in the object store but has not yet been referenced by any tree or commit — it is temporarily unreachable.

2. Concurrently, GC runs its mark phase. It traverses all refs and does not find the new blob (because no commit or tree points to it yet). GC marks the blob as unreachable.

3. GC's sweep phase deletes the blob file.

4. The writer continues: it creates a tree that references the blob hash, then creates a commit referencing the tree, and finally updates HEAD. The commit object is stored successfully, but when something later tries to read that blob — to checkout, diff, or verify integrity — it finds the object missing. The repository is now corrupt.

How real Git avoids this: Git uses a *grace period* (default 2 weeks for loose objects; configurable). GC only deletes objects older than the grace period, so a blob written seconds ago by a concurrent commit is safe. Additionally, Git writes objects atomically (temp file + rename), and the GC process checks object *mtime*: any object modified more recently than the grace period is kept regardless of reachability. For pack-file GC (*git repack*), Git also holds a *gc.pid* lock file that prevents two GC processes from running simultaneously, and the pack creation pipeline only removes loose objects after the new pack index is fully written and fsync'd. Some Git versions also use reference transaction hooks and *gc --auto* throttling to reduce the window during which objects are unreachable-but-needed.